

1. Incident Auto-Categorization Based on Description

Whenever an Incident is created, if the short description or description contains terms like email, outlook, or mailbox, the system should automatically set Category = Application and Subcategory = Email. This should happen only if the user has not manually selected category/subcategory, should work on insert and update, and should not overwrite values set by resolver groups later. Explain how you would implement this, which script type you would use, and how you would avoid overwriting manual changes.

2. Prevent Duplicate Major Incidents

When an Incident is being marked as Major Incident = true, the system should check whether there is already another active Major Incident with the same business service, same assignment group, and active = true. If found, the save should be prevented and a clear error message should be shown. Explain where you would write this validation, why, and how you would ensure it works through UI, API, and import.

3. Auto-Close Child Incidents When Parent is Resolved

A parent Incident may have multiple child Incidents linked through parent_incident. When the parent Incident moves to Resolved, all active child Incidents should be updated to State = Resolved, Close notes = "Resolved as part of parent incident resolution", and Resolution code = "Solved (Permanently)". Only active children should be updated, work notes should mention the parent number, and the script should avoid unnecessary updates. Design the logic and explain which Business Rule timing you would choose.

4. Reopen Count Tracking

Whenever a resolved Incident is reopened, increment a custom field u_reopen_count. The count should increment only when state changes from Resolved or Closed to any active working state. If the record is updated for any other reason, the count must not change. The logic should work whether the record is reopened through UI, API, import, or flow. Write the Business Rule logic and explain how you would compare previous and current values.

5. Mandatory Work Notes for Priority 1 and 2 Updates

If an Incident priority is 1 - Critical or 2 - High, then on every update after creation, work notes must be provided. Comments alone should not be enough. Explain how you would implement this reliably. Would client-side validation be sufficient? Write the script approach you recommend.

6. Dynamic Assignment Group Suggestion Using GlideAjax

On the Incident form, when the caller and category are selected, the system should suggest the most likely assignment group based on a custom mapping

table. It should not directly set the field, only suggest it by showing a message such as “Suggested group: Network Support”. Data must come from the server side and the response should be secure and fast. Design the Client Script, Script Include, and GlideAjax solution.

7. Block Resolution If Open Tasks Exist

An Incident cannot be resolved if it has any related active Incident Tasks. Write a server-side validation for this. Explain how you would efficiently query for related active tasks and what script type and timing you would choose.

8. Auto-Populate Problem from Repeated Incidents

If more than 5 Incidents with the same CI and same category are created within the last 7 days, a Problem record should automatically be created and linked. Only one open Problem should exist for the repeated pattern, duplicates must not be created, the Problem summary should mention the CI and category, and the new Incident should get linked to the existing or newly created Problem. Explain how you would implement this and what script components you would use.

9. Validate Planned Start and End Dates on Change Request

On the Change Request form, planned start date must be in the future and planned end date must be after planned start date. If type = Emergency, start date can be current time or future. Explain how you would split this validation between client-side and server-side, and why both-side validation matters.

10. Auto-Create Incident Task Based on Category

Whenever an Incident with category = Network and priority = 1 is created, the system should automatically create two Incident Tasks: one for router status validation assigned to Network Team, and one for user communication assigned to Service Desk. Tasks should only be created once, updates should not create duplicates, and task short description should include the parent incident number. Design the server-side logic and explain how you would protect against duplicate task creation.

11. Restrict State Transition Based on User Role

Only users with itil_admin role can move an Incident directly from New to Resolved. All other users must move through intermediate working states. Explain how you would implement this rule and whether it should be client-side, server-side, or both.

12. Calculate Business Duration Between Assignment Changes

A custom table tracks Incident assignment history. Whenever assignment group changes, the system should store the previous assignment group, new assignment group, changed by, and the business elapsed time spent with the old

group. Explain how you would capture this in script, what APIs or fields you would use for business duration calculation, and where the logic should sit.

13. Stop Recursive Updates in Business Rules

An Incident Business Rule updates the related Problem record, and a Problem Business Rule updates all linked Incidents, causing recursive updates. Explain how you would redesign this or script defensively to prevent recursion. Include techniques such as condition checks, flags, workflow suppression, or architectural changes.

14. Script Include for Reusable SLA Breach Logic

You need reusable logic to determine whether an Incident is approaching SLA breach. This logic will be used by a Business Rule, Scheduled Job, and UI Action. Inputs should include task sys_id and threshold percentage. Output should include true/false and a breach summary. Design the Script Include, its methods, and how it would be consumed.

15. Enforce Comment on Assignment Group Change

Whenever assignment group changes on an Incident, comments should be mandatory and should explain why reassignment happened. This should apply regardless of how the update happens. Write the script approach and explain how you would detect a real assignment group change and not just a form save.

16. Create a UI Action to Escalate Incident

Create a UI Action called Escalate Incident which is visible only when state is In Progress and priority is 1 or 2. On click, it should set a custom escalation flag, add a work note, reassign the record to Escalation Team, and redirect back to the current record. Write the UI Action logic and visibility condition, and explain what belongs in the condition versus the script.

17. Caller-Specific Incident Limit

A caller should not be allowed to have more than 3 active P1 or P2 Incidents at the same time. Explain how you would enforce this at save time, write the query logic, and mention what indexing or query considerations matter if the Incident table is large.

18. Fetch Related CI Owner on Form Load

When the Incident form loads and a Configuration Item is already selected, display the CI owner name and email in an info message. Explain whether you would use Display Business Rule, Client Script, GlideAjax, or g_scratchpad, and why.

19. Auto-Fill Resolution Notes Template

When an Incident state changes to Resolved, auto-populate close_notes with a

resolution template based on category: Network = “Connectivity restored after validation...”, Email = “Mailbox functionality restored...”, Hardware = “Hardware issue corrected...”. This should happen only if close_notes is empty and should not overwrite manual notes. Explain how you would script this and whether it belongs client-side or server-side.

20. Build a Reusable Validation Engine for Incident Rules

You have multiple Incident validations such as mandatory work notes for P1/P2, blocking resolution if child tasks are open, restricting direct state transition, and preventing duplicate major incidents. Instead of writing separate scattered scripts, design a reusable validation framework using Script Includes and Business Rules. Explain the structure and calling pattern.

21. Detect VIP Caller and Trigger Special Handling

If the caller is marked as VIP, impact should default to High, assignment group should default to VIP Support, a work note should be added automatically, and if the form is being filled manually, the user should see a warning immediately. Explain how you would design this with both immediate UI behavior and reliable backend enforcement.

22. Incident Closure Quality Check

Before an Incident can move to Closed, resolution code must be populated, close notes must be at least 30 characters, and if category = Hardware, close notes must mention an asset tag pattern like AST followed by digits. Design the validation and explain how you would perform the pattern match in script.

23. Scheduled Job to Reassign Stale Incidents

Create a script that runs every hour and reassigns In Progress Incidents to a queue manager if assigned_to is empty, the record has not been updated for more than 2 hours, and priority is 1 or 2. The script should also add a work note indicating auto-reassignment. Write the scheduled job logic and explain how you would query efficiently and what risks you would consider before mass-updating records.

24. Incident Integration Payload Logging

An external system creates Incidents through REST API. You need to log request payload, response status, created incident number, and error message if any, in a reusable and searchable way for later troubleshooting. Explain how you would implement this in ServiceNow scripting terms and whether you would use Business Rules, Scripted REST API, or another design.

25. Conditional Read-Only Behavior Based on State and Role

On the Incident form, if state = Resolved or Closed, all fields should become read-only except for users with itil_admin. Work notes should still remain

editable for itil users. Explain how you would implement this and what should be client-side, ACL-based, UI policy-based, or server-side.

26. Prevent CI Change After Work Has Started

Once an Incident has moved beyond New, the Configuration Item should not be changed unless the user has admin role. Write the validation logic and explain how you would ensure it works for background updates and integrations too.

27. Auto-Link Knowledge Article on Incident Creation

When Incident category and subcategory match a record in a custom mapping table, automatically populate a related knowledge suggestion field with the article reference. This should happen only for new incidents, only if mapping exists, and if multiple matches exist, the highest priority mapping should be selected. Explain how you would script this and what query/table design you would use.

28. Build an Ajax Processor for Caller Validation

When a caller is selected on the Incident form, validate whether the caller is active, has a department assigned, and belongs to supported locations. Return valid/invalid, warning message, and department name. Design the Script Include and Client Script, and explain how you would structure the JSON response from GlideAjax.

29. Add Related Problem Count on Incident Form

On the Incident form, show a field or message that tells the user how many active Problems exist for the same CI. Explain how you would retrieve and display this efficiently, and whether you would use a Display Business Rule, GlideAjax, or derived field update.

30. Bulk Incident Update Utility

A UI Action is required on a custom admin-only module. It should find all active Incidents for a selected assignment group, update priority from 3 to 2, add a common work note, skip incidents already linked to a Major Incident, and return a summary count of updated records. Explain how you would implement this safely and what checks you would include before mass update.

31. Child Task Completion Check Before Parent Closure

A custom parent ticket table has child task records. The parent should move to Closed only if all child tasks are Closed Complete, no child task is Closed Incomplete, and at least one child task exists. Write the script logic and explain how you would handle aggregate validation cleanly.

32. Detect Unauthorized Field Changes

In Incident table, if a non-admin user changes priority, business service, or

impact, the system should log old value, new value, user, and timestamp into a custom audit table. Explain how you would implement this, what Business Rule timing would be suitable, and how you would compare values correctly.

33. Incident Watch List Auto-Management

Whenever assignment group changes, add the new group manager to the watch list, remove the previous group manager from the watch list, do not remove manually added users, and avoid duplicate sys_ids in the watch list. Explain how you would script this watch list management robustly.

34. Smart Reopen UI Action

Create a UI Action called Reopen Incident that appears only when the Incident is Resolved. On click, it should set state to In Progress, increment reopen count, make reopen reason mandatory, and write the reason into work notes. Explain whether all of this should be handled only in the UI Action or also elsewhere, and write the safest design.

35. Service Offering Validation Based on Business Service

When business service is selected, the service offering must belong to that business service. If not, the user should not be able to save. Explain how you would validate this reference relationship and write both client-side and server-side strategy.